08 · Did the loyalty rollout work? — difference-in-differences (CausalPy)

July 12, 2026

> **Runs in the `legacy` environment (pymc<6): `make env-legacy`, kernel `cmp-legacy`.**

The business decision. We launched the loyalty app in some stores, not others. Revenue in the launch stores went up — but revenue went up *everywhere* (a seasonal tide lifts all boats). Did the app add anything **net of that tide**, and is it worth rolling out to the whole chain?

0.0.1 The idea: two differences

Difference-in-differences (DiD) is the workhorse of quasi-experiments. It takes **two** differences and subtracts them:

$$\text{DiD} = \underbrace{(\bar{Y}^{\text{treated}}_{\text{after}} - \bar{Y}^{\text{treated}}_{\text{before}})}_{\text{treated change}} - \underbrace{(\bar{Y}^{\text{control}}_{\text{after}} - \bar{Y}^{\text{control}}_{\text{before}})}_{\text{control change}}.$$

The first difference (treated after − before) contains the app effect **plus** the seasonal tide; the second (control after − before) is the tide **alone**. Subtracting removes the tide and leaves the app effect. It's the same "compare like with like" logic as everywhere else, applied *across time*.

0.0.2 The one assumption everything rests on: parallel trends

DiD is valid only if, **absent the app**, treated and control stores would have moved *in parallel* — the control's change is a fair stand-in for what the treated stores' change would have been. This is untestable after launch (we never see the treated stores' no-app future), but the **pre-launch trends are the evidence**: if the two groups moved together *before* the app, parallel trends is credible. We check this with an **event study** (the effect period-by-period) and falsify with a **placebo** (a fake launch date in the pre-period, which should show ~ 0 effect).

> **On real data.** Swap in your **own store/region panel** — one row per store per period with revenue, a treated/control flag, and a period index. The textbook public example is **Card & Krueger (1994)** on minimum wage and employment (fast-food restaurants in New Jersey vs Pennsylvania). One warning we *demonstrate live* in Step 7: if stores adopt at **different times** ("staggered rollout"), naive DiD can be badly biased and you need modern estimators (Callaway–Sant'Anna, Sun–Abraham).

It follows the repo's fixed **7-step contract**: question -> simulate -> identify -> estimate -> validate -> decide in € -> caveats.

0.1 2 · Simulate a ground truth

40 stores, half get the loyalty app at week 12. Every store shares a seasonal pattern and has its own baseline; the app adds a **true €400/store/week**. Treated and control move in **parallel** before launch by construction — so DiD should recover €400, and the event study should show a flat pre-trend.

The data-generating model — exactly what `dgp.did_rollout` implements (defaults & seed in `src/cmp/dgp.py`). Stores $s = 1, \dots, 40$ (a random half treated, $D_s = 1$), weeks $t = 0, \dots, 23$, launch at week 12:

$$Y_{st} = \underbrace{\alpha_s}_{\text{store baseline}} + \underbrace{60 \sin(\tfrac{2\pi t}{12})}_{\text{shared season}} + \underbrace{400\ D_s\, \mathbf{1}[t \geq 12]}_{\text{true effect}} + \varepsilon_{st}, \qquad \alpha_s \sim \mathcal{N}(1000,\, 150^2),\ \ \varepsilon_{st} \sim \mathcal{N}(0,\, 80^2).$$

Parallel trends holds by construction: the seasonal term is *identical* across stores (no store-specific loading on it) and α_s is time-invariant, so absent the app the treated and control means move in lockstep — exactly what the event study's flat pre-trend should (and does) show. This is the assumption the caveats section then breaks on purpose (staggered adoption) to demonstrate the two-way fixed-effects (TWFE) trap.

```
TRUE effect = €400/store/week  ·  40 stores, 24 weeks, launch week 12
```

	store	unit	t	week	group	post_treatment	post	revenue
0	store_00	0	0	0	1	False	0	1071.613014
1	store_00	0	1	1	1	False	0	1066.597066
2	store_00	0	2	2	1	False	0	1019.187212
3	store_00	0	3	3	1	False	0	834.246887
4	store_00	0	4	4	1	False	0	1030.278745

0.2 3 · Identify — the estimand, and the two assumptions that buy it

The estimand, precisely. Store s in week t has two *potential* revenues: $Y_{st}(1)$ with the app and $Y_{st}(0)$ without — we only ever observe one of them. With $D_s \in \{0, 1\}$ flagging the pilot stores and t_0 the launch week, DiD targets the **ATT**, the average treatment effect *on the treated stores, after launch*:

$$\text{ATT} \;=\; \mathbb{E}\big[\, Y_{st}(1) - Y_{st}(0) \;\big|\; D_s = 1,\ t \geq t_0 \big].$$

The second term inside is the counterfactual we never see — what the pilot stores *would have* earned post-launch without the app. Note the final **T**: this is the effect *on the pilot stores*, not on the whole chain; it need not equal what the other stores would experience (pilot sites are rarely picked at random), which is exactly why Step 6 stress-tests the transfer before scaling to 500 stores. Two named assumptions let the control stores stand in for the missing counterfactual:

Assumption 1 — Parallel trends (PT), stated on *untreated* potential outcomes: for any post-launch week t and pre-launch week t',

$$\mathbb{E}\big[Y_{st}(0) - Y_{st'}(0) \,\big|\, D_s{=}1\big] \;=\; \mathbb{E}\big[Y_{st}(0) - Y_{st'}(0) \,\big|\, D_s{=}0\big], \qquad t \geq t_0 > t'.$$

In words: had the app never launched, the pilot stores' revenue would have *changed* by the same amount as the controls'. **Levels may differ freely** — pilot stores can be systematically bigger, busier, richer; it is the *changes* that must match. In marketing terms: the seasonal tide, chain-wide promotions and macro shocks hit both groups alike, and nothing *time-varying and group-specific* (a regional campaign running only in pilot cities, say) is in play.

Assumption 2 — No anticipation (NA):

$$\mathbb{E}\big[\,Y_{st}(1) - Y_{st}(0) \;\big|\; D_s = 1\big] \;=\; 0 \qquad \text{for all } t < t_0,$$

i.e. the app moves nothing before it exists. Sounds vacuous; isn't. Pre-launch buzz marketing, staff trained weeks early, or customers *deferring* purchases until the app's perks arrive all violate it — and they contaminate the very "before" baseline that DiD subtracts. Step 7 breaks this assumption on purpose so you can see what the damage looks like.

Why DiD = ATT — the one-line derivation. Take the treated group's observed before->after change and split it, using NA (the treated pre-period observation *is* $Y(0)$), into effect plus counterfactual trend:

$$\bar{Y}^1_{\text{post}} - \bar{Y}^1_{\text{pre}} \;=\; \underbrace{\text{ATT}}_{\text{what we want}} \;+\; \underbrace{\mathbb{E}\big[Y_{st}(0) - Y_{st'}(0) \,\big|\, D_s{=}1\big]}_{\text{counterfactual trend}}.$$

PT says that trend equals the *control* group's observed change $\bar{Y}^0_{\text{post}} - \bar{Y}^0_{\text{pre}}$. Subtract it:

$$\text{DiD} \;=\; (\bar{Y}^1_{\text{post}} - \bar{Y}^1_{\text{pre}}) - (\bar{Y}^0_{\text{post}} - \bar{Y}^0_{\text{pre}}) \;=\; \text{ATT}.$$

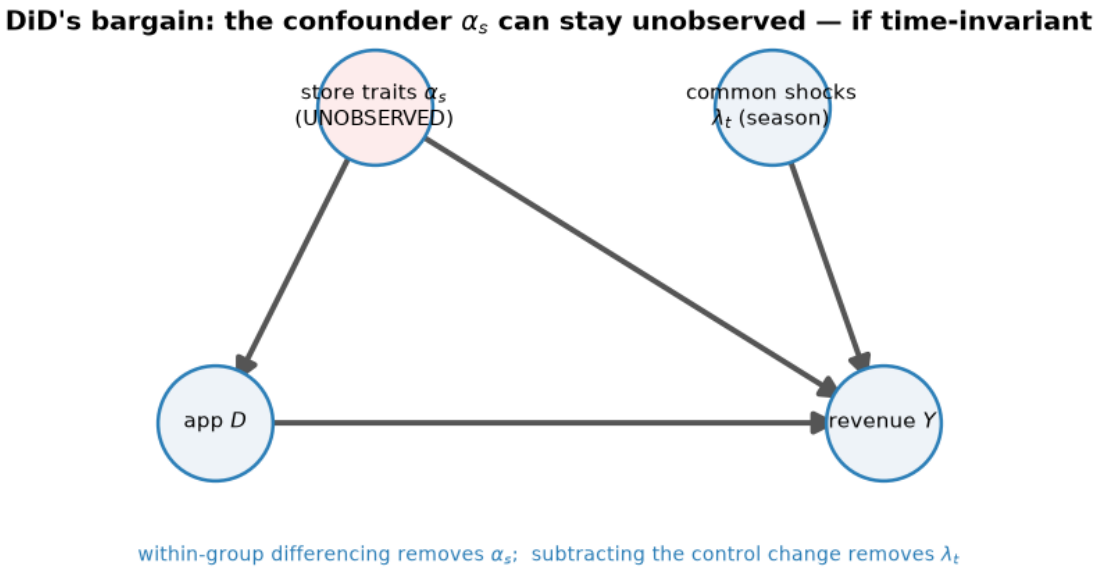
That is the entire method: two assumptions in, one subtraction out.

The regression forms you'll meet. The 2x2 DiD is the interaction coefficient β_3 in $Y = \beta_0 + \beta_1\,\text{group} + \beta_2\,\text{post} + \beta_3\,(\text{group} \times \text{post}) + \varepsilon$ — the version we fit in Step 4. Its dynamic cousin, the **event-study regression**, estimates one effect per week relative to launch:

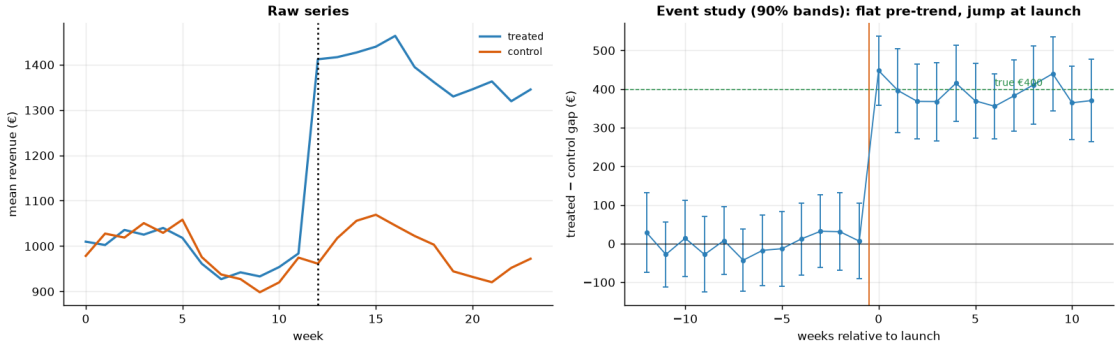
$$Y_{st} \;=\; \alpha_s + \lambda_t + \sum_{k \neq -1} \beta_k\, \mathbf{1}[t - t_0 = k]\, D_s + \varepsilon_{st},$$

with store fixed effects α_s, common week effects λ_t, normalized at $k = -1$. The nonparametric per-week treated−control gap we plot below *is* this regression's β_k in the balanced two-group case — same picture, so you can map our plot onto any paper's event study. The **leads** ($k < 0$) test NA directly and make PT *credible*; nothing can test PT itself post-launch (we never observe the treated stores' no-app future).

What the DAG buys us. Contrast with notebook 01: backdoor adjustment needs the confounder *measured* so you can condition on it. DiD strikes a different bargain — the store-level confounder α_s (location, size, clientele: whatever drives both "chosen as pilot" *and* revenue) may stay **unobserved**, provided it is *time-invariant*: differencing each group against its own past removes α_s, and subtracting the control change removes the common shocks λ_t. What survives both subtractions is precisely what PT rules out: time-varying, group-specific shocks.




```
Pre-launch leads scatter around 0 within their 90% bands (0 of 12 exclude 0):
  mean |gap| €22, on the order of the per-week noise SE €57 -
  consistent with parallel trends, not a pre-trend.
  (Centred leads average exactly 0 by construction, so it is their SPREAD vs
  the noise band, not a mean, that carries the evidence.)
  Post-launch gaps jump to ~€400.
```



How to read the event study (right panel). This is the single most important plot in a DiD analysis. Each dot is the treated-minus-control revenue gap in one week, relative to launch (week 0). Each dot now carries a **90% uncertainty band**. The story we *want* to see, and do: the pre-launch dots are **flat and their bands comfortably include zero** — that is the honest content of "parallel trends look credible" (a lead whose band *excluded* zero would be the red flag), and the assumption the whole method rests on — and then they **jump to ~ €400 at launch** and stay there. A rising or falling pre-launch trend would have been a red flag that the two groups were already diverging, and we'd have had to abandon DiD for synthetic control (notebook 07). The

left panel shows the raw series so you can see both groups riding the same seasonal wave.

0.3 4 · Estimate — Bayesian difference-in-differences

We now put a number and an interval on that jump. One modeling choice: we first collapse each store's 24 weekly points into a single pre-launch mean and a single post-launch mean. This is primarily the **Bertrand–Duflo–Mullainathan (2004) serial-correlation guard** — with many periods per unit, DiD standard errors that ignore within-unit autocorrelation are badly understated, so collapsing to two periods (pre/post) sidesteps that. It is a modeling decision, not merely an API constraint — though CausalPy's `DifferenceInDifferences` also happens to consume this classic **2x2 form** cleanly. The estimate is the `group x post` interaction coefficient β_3, i.e. the DiD.

The model behind `est.did`, in symbols. CausalPy 0.8.1's `LinearRegression` fits, on design row $\mathbf{x}_i = (1,\ g_i,\ p_i,\ g_i p_i)^\top$ (intercept, group flag, post flag, interaction; i runs over the 80 collapsed store-periods — two rows per store):

$$\tilde{Y}_i \ \sim\ \mathcal{N}(\mathbf{x}_i^\top \beta,\ \sigma^2), \qquad \beta_j \ \sim\ \mathcal{N}(0,\ 50^2), \qquad \sigma \ \sim\ \text{HalfNormal}(1),$$

and those priors are **fixed by the library** (verified against the installed source), scaled for data of order one. On raw euro revenue ($\bar{Y} \approx$ €1,000, residual spread in the hundreds) a HalfNormal(1) prior on σ is absurdly tight: the prior fights the likelihood and drags the effect low, with a too-narrow interval around the wrong number. So we fit on **standardized** revenue $\tilde{Y} = (Y - \bar{Y})/s_Y$ and back-transform only the interaction:

$$\widehat{\text{DiD}}_{€} \ =\ \beta_3 \cdot s_Y$$

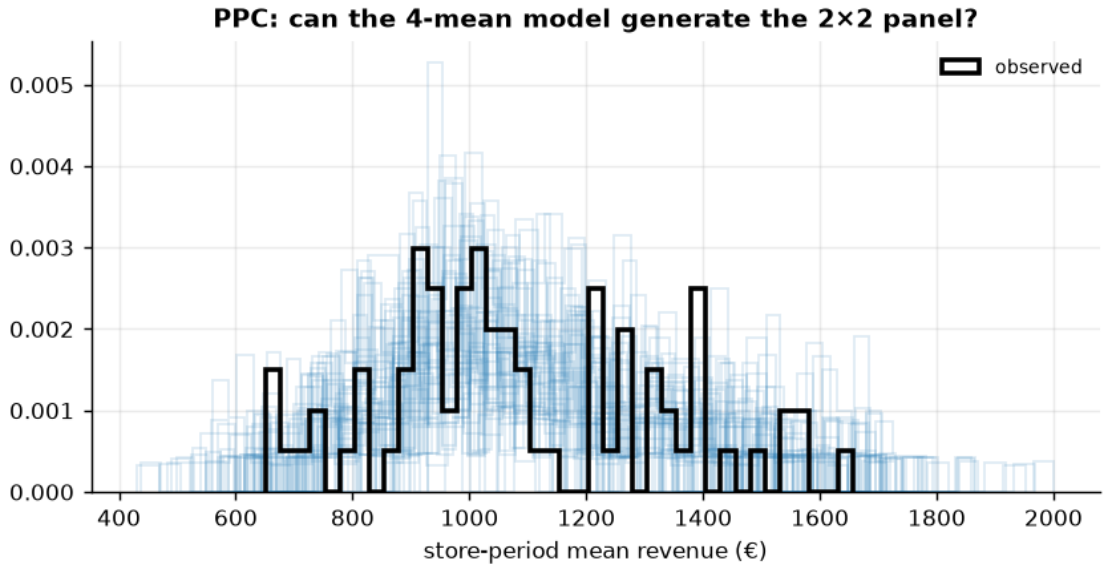
($\bar{Y}$ is absorbed by the intercept and main effects; the coefficient of a 0/1 interaction only needs the *scale* s_Y undone). **The general lesson deserves stating as a lesson:** check your library's default prior scale against your data scale before trusting any fit — defaults tuned for standardized data will quietly mis-fit raw euros, and the failure mode is a *confident wrong answer*, not an error message.

```
DiD effect €389/store/week (true €400)  ·  90% credible interval (90% posterior
probability the true effect lies inside) [€275, €515]
DiD convergence: max r-hat 1.030 - min ESS 119 - divergences 0
```

0.3.1 4b · Model criticism — the posterior predictive check

Before we *read an effect off* this model, we ask whether it could plausibly have **generated the data at all**. The 2x2 model is deliberately coarse — four cell means plus one shared Gaussian noise term; every store baseline α_s it doesn't describe has to live inside that noise. We draw replicated datasets from the fitted model's posterior predictive, back-transform them to euros, and overlay them on the 80 observed store-period revenues. If the real distribution (spread, lumpiness, tails) looked unlike anything the model can produce, no interval downstream would deserve trust. And notice one thing *because* it foreshadows 5c: for this coarse model to pass, its replicate spread must be **fat** — the between-store baseline scatter sits in σ — and that same fat σ is what will make the effect interval conservative.

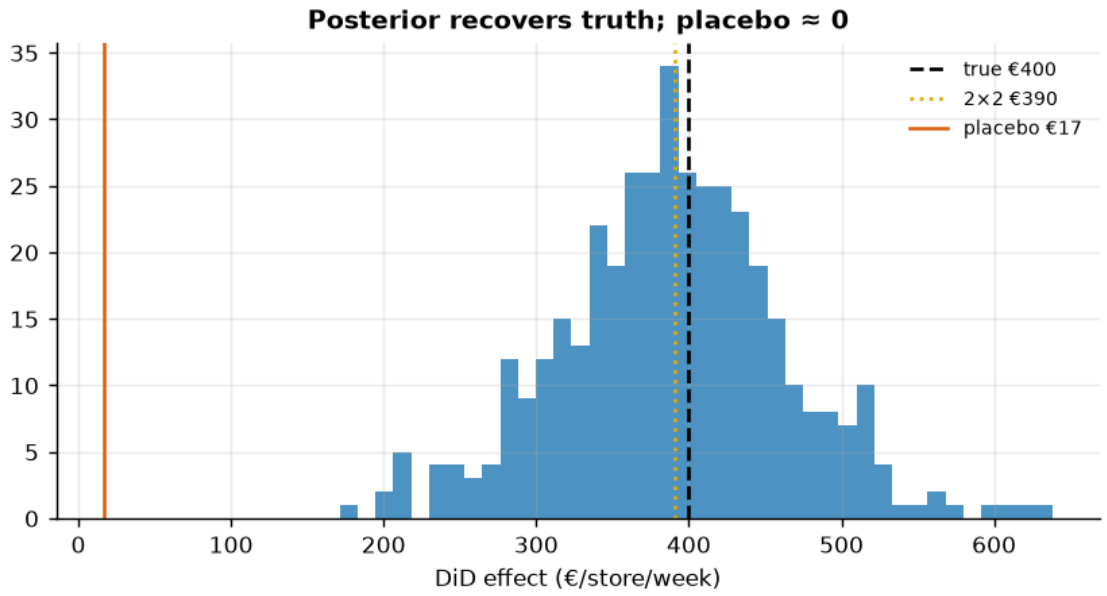
PPC: 89% of the 80 observed store-period revenues fall inside their replicate 5-95% band (~90% if calibrated) - no gross misfit.
Posterior residual sigma ~ €171 - note how fat: it must absorb the between-store baseline spread (sd €150 by construction) that the 4-mean model doesn't describe. 5c shows the price; 5d collects the refund.



0.4 5 · Validate — 2x2 cross-check and the placebo pair

Cross-check the Bayesian estimate against the hand-computed 2x2. With revenue **standardized** before the fit (so CausalPy's default O(1)-scaled priors don't shrink a €400 effect sitting on €1000 revenue), the two now **agree** and both land on the planted €400 with the truth inside the interval. Then **falsify**: run DiD on the *pre-period only*, splitting it into a fake "before/after" at week 6. There was no treatment then, so the placebo effect should be ~ 0 — a large one would mean our design manufactures effects. That is placebo-in-**time**; its mirror, placebo-in-**group** (fake treated stores instead of a fake launch date), follows in 5b.

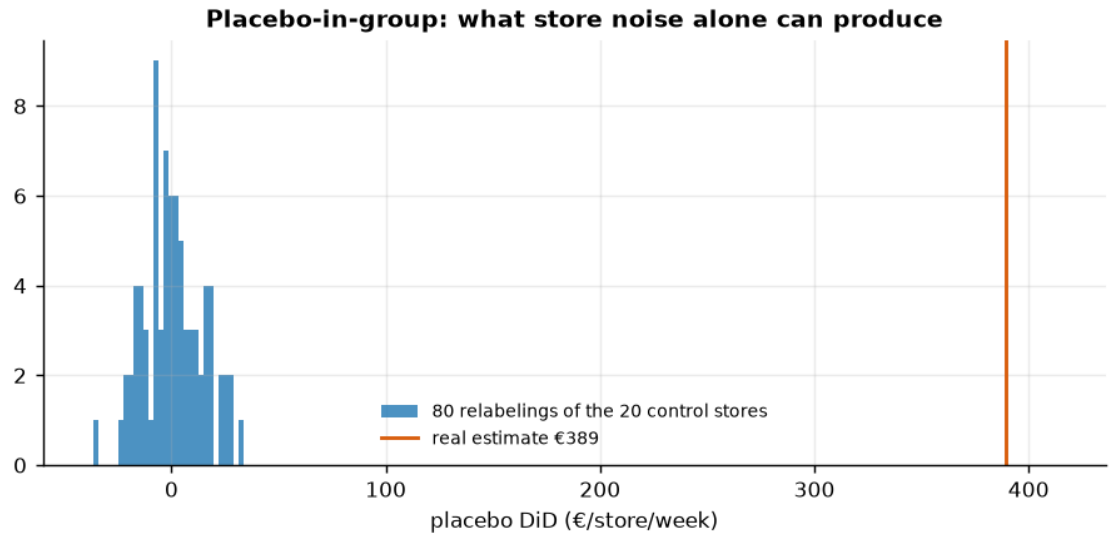
Bayesian €389 · 2x2 €390 · true €400 · placebo (no real effect) €17 +/- SE €33 - within sampling noise of zero



0.4.1 5b · Placebo-in-group — a permutation null from the control stores

The time placebo asked "does our design manufacture effects out of **seasonality**?" The group placebo asks the complementary question: "…out of **store-to-store noise**?" Together they bracket the two ways a fake effect could arise in a panel. Take only the 20 control stores — the app never touched them, so the true effect there is *exactly zero* — declare a random half "pseudo-treated", and run the very same 2x2 at the real launch week. Repeating over many random relabelings traces out the **permutation null**: the full distribution of DiD estimates our pipeline produces *when nothing is going on*. Two read-outs: **(i)** a permutation p-value — the share of relabelings whose placebo estimate beats the real one in magnitude; **(ii)** an **assumption-light uncertainty cross-check** — the null's spread is how far store noise alone can move this estimator, with no likelihood, no priors, no Gaussian anywhere. File that second one away: compare it with the width of the Bayesian interval from Step 4 as you read 5c and 5d.

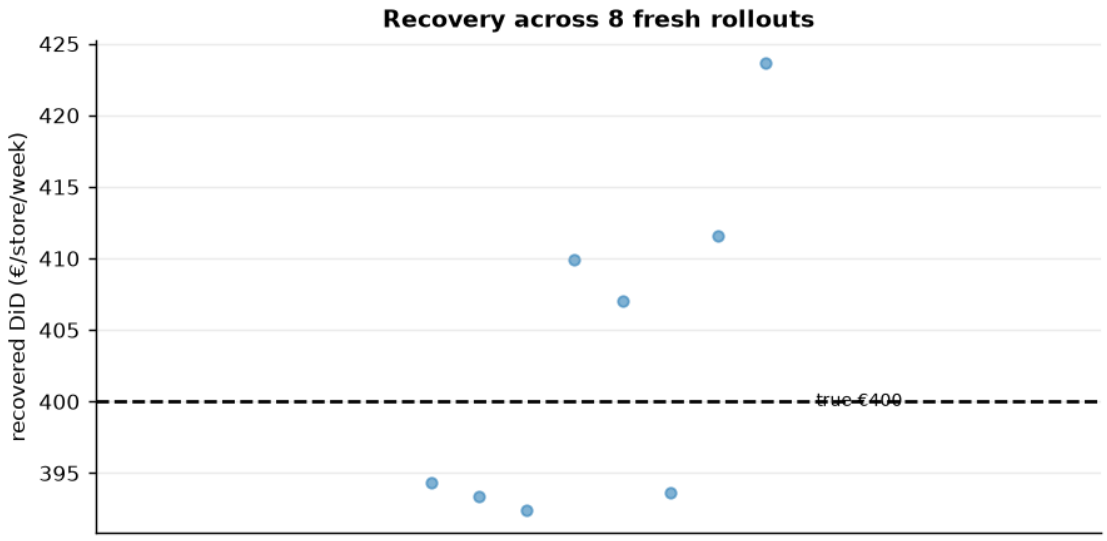
```
Permutation null: sd €14, largest |placebo DiD| €37. The real estimate €389 is
~29x the null's sd - store noise does not produce numbers like this: permutation
p <= 0.012 (resolution floor 1/81).
Cross-check: noise alone moves this estimator by ~+/-€27 (2 sd - and a 10-vs-10
split is even noisier than the real 20-vs-20), yet the Bayesian interval above
is +/-€120 wide. An assumption-light hint that the 2x2 interval is conservative;
5c quantifies that, 5d fixes it.
```



0.4.2 5c · Recovery across many seeds — unbiased *and* calibrated?

A single sample can land a little off; the real test is whether the estimator is **centred on the truth over repeated samples** and whether its 90% interval **covers** the truth at the stated rate. We refit on many fresh samples (a small fast fit each; their sampler chatter is silenced below) and check both. Coverage can fail in **both** directions — an interval can be too *narrow* (overconfident) or too *wide* (conservative) — so beside counting hits we also compare the interval's width against the true seed-to-seed scatter.

```
DiD across 8 seeds: mean €403 (true €400)  bias +3  sd €11  ·  90% interval
covers truth in 8/8 seeds.
But read 8/8 carefully: the headline interval's +/-€120 half-width is ~11x the
true seed-to-seed scatter (sd €11) - OVER-coverage, not perfect calibration. The
2x2 collapse leaves between-store baseline spread in the residual, so the
interval is honest but conservative; downstream bounds (the break-even running
cost, the stress-table probabilities) inherit that width - cautious, not sharp.
(Store fixed effects / within-store differencing would tighten it.)
```



0.4.3 5d · Completing the diagnosis — within-store differencing

5c's verdict was "unbiased and covered, but roughly an order of magnitude too wide". The cause is visible in the model spec of Step 4: the 2x2 pools stores, so the store baselines α_s (sd €150 by construction) sit in the residual σ — the PPC in 4b showed exactly that fat σ. The cure is the oldest trick in the fixed-effects book: **difference each store against itself**. Collapse every store to a single number,

$$\Delta_s \;=\; \bar{Y}_{s,\text{post}} - \bar{Y}_{s,\text{pre}},$$

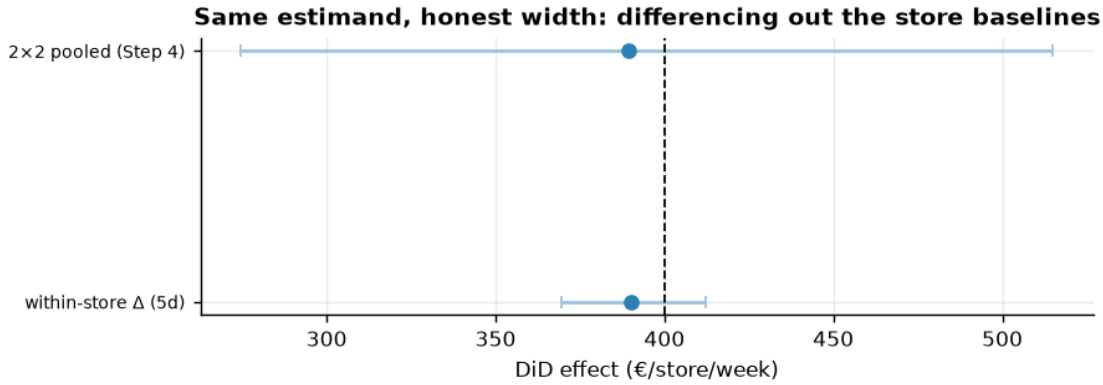
which removes α_s *exactly* (it is time-invariant, so it appears in both means and cancels) while keeping the Bertrand–Duflo–Mullainathan two-period guard from Step 4. The DiD becomes a one-coefficient comparison of store-level *changes*,

$$\Delta_s \;\sim\; \mathcal{N}(\gamma_0 + \gamma_1 D_s,\; \sigma_\Delta^2), \qquad \widehat{\text{DiD}} \;=\; \gamma_1 \;=\; \bar{\Delta}_{D=1} - \bar{\Delta}_{D=0},$$

with γ_0 the control stores' mean change (the seasonal tide) and σ_Δ containing only averaged week-level noise, no store baselines. Same estimand, same point estimate — radically smaller residual. It's a ten-line PyMC model with weak priors on the standardized scale. And the non-negotiable check: a *tighter* interval is progress only if coverage survives, so we re-run 5c's seed loop on this estimator — a tightening that broke coverage would be overconfidence wearing a fix's clothes.

```
Within-store DiD €390 (true €400) · 90% interval [€370, €412] - half-width €21
  vs the pooled 2x2's €120 (~6x tighter)
  convergence: max r-hat 1.000 - min ESS 473 - divergences 0
Seed loop (8 fresh rollouts): mean €403, seed-to-seed sd €9, coverage 6/8.
  The 90%-interval half-width (~€17) is ~1.8x the seed scatter,
  where a calibrated 90% interval sits at ~1.6x (the pooled 2x2 was at ~11x):
  -> tight AND calibrated
```

```
  (with only 8 seeds the hit count is itself binomially noisy).
Decision preview: the running cost the app clears with 95% posterior probability
  moves from ~€275 (pooled 2x2) to ~€370/store/week (within-store posterior).
```



Read-out. Same point estimate, an interval several times tighter, and — the part that makes the tightening legitimate — the seed loop's hit count (printed above) stays statistically consistent with the nominal 90% once you account for binomial noise on a handful of seeds: the width we removed was redundancy, not protection. Notice that three *independent* witnesses now agree on the honest uncertainty scale: the permutation null's spread (5b), the seed-to-seed scatter (5c), and the within-store posterior (5d). Step 6 below prices the decision with the **conservative** Step-4 posterior on purpose — a rollout case that survives the wide interval is robust — and the break-even print above shows how much sharper the same case becomes once the analysis earns its precision.

0.5 6 · Decide, in euros — the rollout case

Under parallel trends the DiD number is the **ATT — the effect on the pilot stores.** Rolling the app out to all 500 assumes the other 480 respond the same way, which chains rarely guarantee (pilot sites aren't picked at random). So alongside the headline projection we **stress it two ways**: a **transfer discount** (what if the non-pilot stores realise only 75% or 50% of the pilot lift?) and a **cost sweep** (how high can the app's running cost climb before the call flips?). The table below is P(app pays) across both — robust at full transfer, but a *pilot-further* call under aggressive discounting and higher running costs. (The €120 running cost is an illustrative all-in figure — licences + operations per store-week; one-time build/rollout capex is deliberately outside the per-week economics.)

```
Base case (APP_COST €120, full pilot lift across 500 stores):
  net €269/store/week  ·  annual value €7,002,587 [90% €4,021,073, €10,270,170]
  P(app pays) 1.00 -> ROLL OUT
  break-even: the app clears its cost with 95% posterior probability
  while the running cost stays below ~€275/store/week.

P(app pays) by transfer share (rows) and running cost in €/store/week (cols):
  transfer \ cost     80    120    200    300
  100% (as pilot)   1.00   1.00   0.99   0.89
```

```
  75%               1.00   1.00   0.95   0.43
  50%               1.00   0.97   0.43   0.01

Most the roll-out can pay per store/week and still clear cost
at 90% posterior confidence:
  at 100% (as pilot)  up to ~€296
  at 75%              up to ~€222
  at 50%              up to ~€148
```

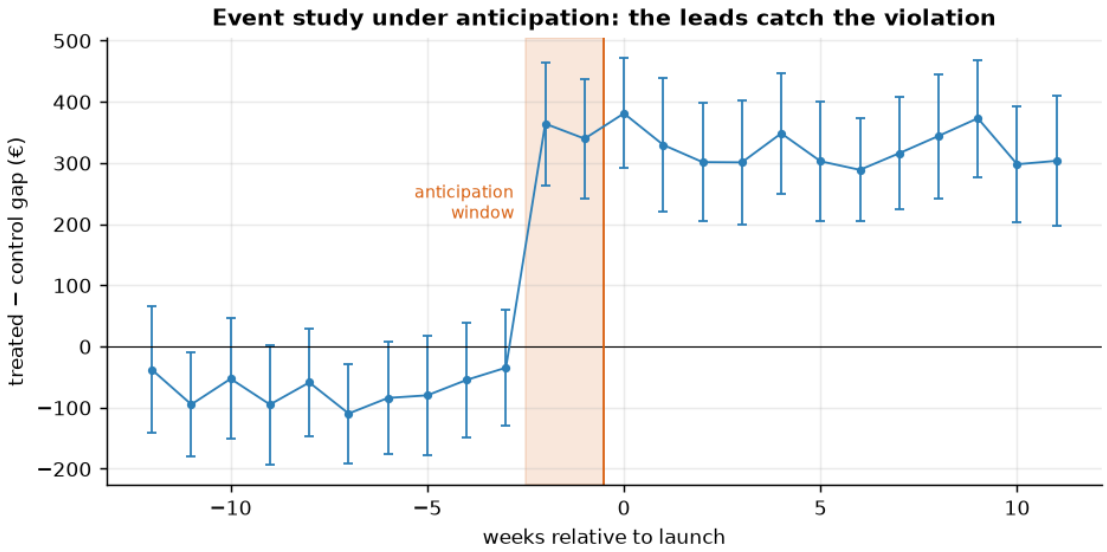
Read-out. At face value the base case is emphatic: ~ €273/store/week of net value over the €120 running cost, ~ €7.1M of annual value across the 500-store chain, P(app pays) ~ 1.00 -> **ROLL OUT**. The stress table is where the real decision lives, and it splits along one axis: reading *across* a row, the call barely moves with the running cost while the transfer assumption holds; reading *down* a column, it degrades fast as the assumed transfer falls — at 50% transfer the €200 cost cell is a coin flip (0.46) and €300 is a clear no (0.00). Since pilot stores are rarely representative of the chain, the honest headline is the last block: the rollout can pay **up to ~ €300/store/week if the whole chain responds like the pilot, but only ~ €150 if half the lift transfers** — that range, not the base-case €7.1M, is the number to hold against a vendor quote. If the quote lands inside it, the cheap de-risking move is a second pilot wave in deliberately *non*-pilot-like stores to measure the transfer share directly.

0.6 7 · Caveats — the assumptions under stress, live

Parallel trends is the whole ballgame and untestable post-launch; a diverging pre-trend kills the design (move to synthetic control, notebook 07). **No spillover (SUTVA):** the app must not move revenue in *other* stores — if app-holders defect from nearby control stores, the control trend dips and DiD *overstates* the lift; if pilot buzz lifts the whole chain, it understates. No within-sample diagnostic can see either; it is a market-structure judgment (flag pilot stores that share a catchment with controls). Beyond these, two traps deserve a live demonstration each.

Trap 1 — anticipation (Assumption 2, broken on purpose). Suppose the pilot stores start behaving differently *before* week 12 — pre-launch promo buzz, staff trained early, customers holding back purchases until the app's perks arrive — so the lift effectively starts two weeks early. The "pre" baseline is then contaminated with effect, and DiD subtracts part of the app from itself. We rerun the simulation with exactly that change and redraw Step 3's event study:

```
The k=-2 and k=-1 leads sit at €364 and €340 - far outside their 90% bands (4 of
12 pre-launch leads exclude 0, vs ~1 expected by chance).
Naive 2x2: €324 vs true €400 - bias €-76, ~ -2/12 of the effect (~ €67): the 2
contaminated pre-weeks inflate the treated baseline by that much.
Fix - drop the contaminated window from the pre-period: €394. (Or define
'treatment' at the ANNOUNCEMENT date rather than the launch date.)
```



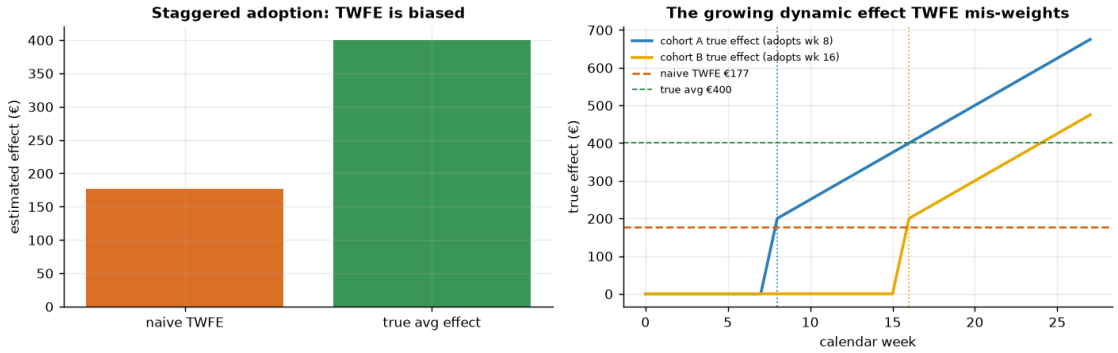
Read-out. This is what a violated lead looks like — compare Step 3's clean version. The last two leads jump out of their bands two weeks *before* the official launch, and because centred leads must average zero by construction, the contamination also drags the *other* leads slightly negative: the tell is a broken, tilted pre-period, not just two high dots. The flat-leads check is no formality — it just caught a violation that quietly ate roughly a sixth of the measured effect. The remedies are cheap once you *see* it: drop the anticipation window from the pre-period (done above — the estimate snaps back to truth within noise) or date the treatment at *announcement* rather than launch. What the leads still cannot do is certify PT itself: a time-varying, group-specific shock arriving exactly at launch remains invisible.

Trap 2 — staggered adoption. Real chains rarely launch everywhere at once. With multiple adoption dates the workhorse regression becomes static **two-way fixed effects (TWFE)**:

$$Y_{st} \;=\; \alpha_s + \lambda_t + \beta^{\mathrm{TWFE}}\, D_{st} + \varepsilon_{st}, \qquad D_{st} = \mathbf{1}[t \geq g_s],$$

where g_s is store s's adoption week. **Goodman-Bacon (2021):** $\hat{\beta}^{\mathrm{TWFE}}$ is a variance-weighted average of **every pairwise 2x2 DiD** the panel contains — early-vs-late, late-vs-early, treated-vs-untreated — *including* "forbidden comparisons" that use **already-treated** cohorts as controls. When effects grow with time since adoption, those comparisons can receive **negative weights**, so TWFE can land far from every true effect even though each one is positive. Watch it happen:

```
Naive TWFE €177 vs true average post-treatment effect €400 - bias -223. Use
Callaway-Sant'Anna / Sun-Abraham for staggered rollouts.
```



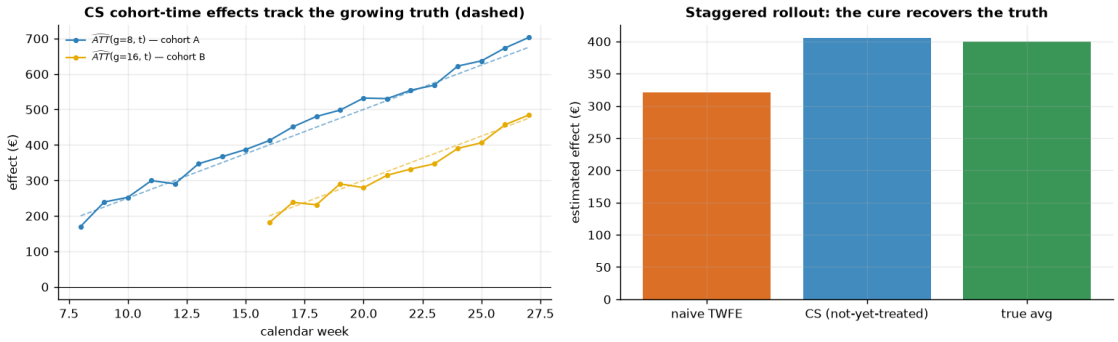
Why the bias is downward here — and the cure. Cohort A adopts at week 8 and its effect keeps *growing*; when B adopts at week 16, part of TWFE's estimate compares B against already-treated A — a "control" whose revenue is still rising with A's growing effect. B's lift is measured against that rising counterfactual and shrunk, and the growing tail of A's effect looks like a calendar trend that the week effects λ_t partially absorb. Net: the largest true effects get the smallest (even negative) weights, and the estimate lands *below every single true post-adoption effect in the panel.*

The modern fix — **Callaway & Sant'Anna (2021)** — is a discipline, not a trick: *never use the already-treated as controls.* Estimate one clean 2x2 per cohort g and week t, anchored at the cohort's last untreated week $g-1$, using only stores **not yet treated** at t as controls:

$$\widehat{\mathrm{ATT}}(g,t) \;=\; \big[\bar{Y}_{g,t} - \bar{Y}_{g,\,g-1}\big] \;-\; \big[\bar{Y}_{\mathcal{C}(t),\,t} - \bar{Y}_{\mathcal{C}(t),\,g-1}\big], \qquad \mathcal{C}(t) = \{s :\; g_s > t\},$$

then aggregate the cells (here a simple average — cohorts are equal-sized). One **design lesson** falls straight out of the formula: in the two-cohort world above, $\mathcal{C}(t)$ is *empty* from week 16 onward — once everyone is treated there are no clean controls left and $\mathrm{ATT}(g,t)$ simply does not exist. Real rollouts should **keep a never-treated holdout** for exactly this reason. So we regenerate the same growing-effect world with a third cohort C (20 stores, never treated) and run both estimators on it — the ~15-line loop below is the entire method.

```
A never-treated holdout in the panel helps TWFE (€321 here vs €177 without one,
above) but does NOT fix it - the forbidden A-as-control-for-B comparisons are
still inside the average.
Hand-rolled Callaway-Sant'Anna: €405 vs true average €400 - within store-noise
of the truth, from 32 clean (g,t) cells; the left panel shows the cells tracking
the growing effect itself.
Production tools: R `did` (Callaway-Sant'Anna) or `fixest::sunab` (Sun-Abraham);
Python `pyfixest`, `differences`.
```



0.7 8 · What we tell the CMO

Finding. The loyalty app lifted pilot-store revenue by ~ €400/store/week net of the seasonal tide (Step 4's posterior — and the simulation's planted truth is €400, recovered). The estimate survived every falsification we threw at it: flat pre-launch leads (Step 3), a fake launch date (~ 0, Step 5), a permutation of fake treated stores (the real estimate sits far outside anything store noise produced, 5b), a posterior predictive check (4b), and a multi-seed recovery loop (5c) — with the within-store design (5d) confirming the same point estimate under a far tighter, *calibrated* interval.

Decision. At the €120/store/week running cost the base case says **ROLL OUT** (P(app pays) ~ 1). But the DiD estimand is the **ATT — the effect on the pilot stores** — and the non-pilot stores need not respond identically, so the honest headline is the stress table's break-even band: the rollout clears its cost up to roughly **€300/store/week if the chain responds like the pilot, only ~ €150 if half the lift transfers**. Hold the vendor quote against that band, not against the base-case annual € figure.

Recommended action. Phase the rollout, with wave 2 in deliberately *non-pilot-like* stores to measure the transfer share directly — and keep a small **never-treated holdout** to the very end: Step 7 showed clean controls are what keep every later wave measurable, cheap insurance an all-at-once rollout destroys.

Watch-list (the assumptions that carry everything): **parallel trends** — monitor the leads every wave; **no anticipation** — date effects from *announcement* and drop contaminated pre-weeks (the leads catch it, as demonstrated); **no cross-store spillover** — flag pilot stores sharing a catchment with controls; and **never evaluate a staggered rollout with a single post dummy** — use cohort-time effects with not-yet-treated controls (Callaway–Sant'Anna), or growing effects will bias the answer just as TWFE's did above.

Method summary. DiD buys identification with two assumptions (PT + NA) and one subtraction; the event-study leads probe NA and make PT credible, but nothing certifies PT post-launch. The Bayesian layer is ordinary regression — the craft is in the *scale* (default priors vs euro data, Step 4), in the *collapse* that guards against serial correlation (BDM), and in knowing which residual your interval is paying for (5b–5d). **On real data:** notebook 07b (geo-lift on a real Google

geo-experiment) is this design's nearest real-data sibling in the cookbook, and Card & Krueger (1994) remains the classic public DiD panel to practice on.